

FMFSTLT Architecture Walkthrough

Implemented TURBOTEST-Style Baseline Pipeline and Planned Foundation Model

Repo root: /mnt/e/fmfstlt

Date: May 1, 2026

Purpose of this report: document, in one place, the exact data flow, tensor shapes, preprocessing stages, and model architectures that already exist in the repository, and then define the next-stage foundation-model architecture with the same level of detail.

Contents

1	Reading Guide	3
2	Part I: Implemented Baseline Reproduction	3
2.1	High-Level End-to-End Flow	3
2.2	What the Original Input Looks Like	4
2.3	Step 1: Raw Shard Construction	5
2.4	Step 2: Train-Only Normalization	6
2.5	Step 3: Deterministic UUID-Level Train/Val Split	6
2.6	Step 4: Stage 1 Window Materialization	7
2.7	Step 5: Implemented Stage 1 Regressor	8
2.8	Step 6: Implemented Stage 2 Dataset Builder	9
2.9	Step 7: Implemented Stage 2 Transformer	12
2.10	Implemented Baseline Summary in One Page	15
3	Part II: Proposed Foundation Model Architecture	16
3.1	Why We Move Beyond the Baseline	16
3.2	Design Goal for the New Model	16
3.3	Fixed Implementation Scope for the Foundation Model	16
3.4	Implementation Decisions Before Coding	16
3.5	Proposed Input Representation	17
3.6	Exact Proposed Foundation Encoder	18
3.7	Proposed Pretraining Objectives	18
3.8	Downstream Heads for the Foundation Model	19
3.8.1	Head 1: Early stopping	19
3.8.2	Head 2: Final throughput regression	20
3.8.3	Head 3: Speed-grade classification	21
3.9	Full Proposed Foundation-Model Flow	21

3.10 Exact Input and Output Contracts for the Proposed Model	22
4 Implemented Baseline vs Proposed Foundation Model	23
5 What Is Already Implemented vs What Comes Next	23
6 Final Summary	23

1 Reading Guide

This report is intentionally split into two parts.

Part I is the *implemented baseline reproduction*. Everything in that part is derived from code that already exists in the repository, primarily:

- `scripts/build_exact_public_shards.py`
- `scripts/compute_exact_public_train_stats.py`
- `scripts/materialize_normalized_exact_public_shards.py`
- `scripts/make_stage1_uuid_split.py`
- `scripts/build_stage1_regression_windows.py`
- `scripts/train_stage1_xgboost.py`
- `scripts/score_stage1_xgboost.py`
- `scripts/build_stage2_transformer_dataset.py`
- `scripts/train_stage2_transformer.py`

Part II is the *planned next architecture*: the foundation model proposed for the project. That part is not an existing implementation yet. It is an exact design specification for the next model we intend to build.

Frozen baseline scope. This baseline reproduction is frozen at the $\epsilon = 10$ operating point. The Pareto sweep across $\epsilon \in \{5, 15, 20, 25, 30, 35\}$ is intentionally not reproduced in this work due to a discovered XGBoost version mismatch in Stage 2 dataset materialization for those epsilon values; rebuilding them is out of scope. All baseline comparisons in this report refer to the $\epsilon = 10$ line.

2 Part I: Implemented Baseline Reproduction

Scope statement. Part I documents the implemented baseline pipeline and treats $\epsilon = 10$ as the frozen internal reproduced baseline. Multi-epsilon baseline rebuilding is not part of the remaining project work.

2.1 High-Level End-to-End Flow

1. Exact public CSV exports

Source tables come from `exports_exact_public`. Each CSV row represents one 100 ms bucket of one speed test. The row carries 13 measurement features, test identifiers, and the final-throughput target that the pipeline tries to recover early.



2. Raw tensor shards

Rows are grouped by UUID and placed on a fixed 100-bucket axis. The output is a dense tensor `x: [N, 100, 13]` plus `bucket_mask: [N, 100]`. Missing buckets are filled by forward propagation of the latest observed bucket. These shards are saved under the raw-shard artifact tree.



3. Train-only normalization

Feature means and standard deviations are computed on the train split only. The same statistics are then applied to train, validation, test, and robustness traces. The normalized outputs become the reusable base representation for every later stage.



4. UUID split and Stage 1 window materialization

Train UUIDs are split into train and validation sets. For each decision point, the pipeline extracts a lookback window of shape `[20,13]` and then flattens it to `[260]`. These fixed-width vectors are saved as Stage 1 regression examples.



5. Stage 1 XGBoost regressor

Stage 1 consumes the flattened `[260]` vectors and predicts one scalar: the final throughput in Mbps. The trained model is saved under `stage1_xgboost_full_windows`.



6. Stage 1 scoring

The trained Stage 1 model is applied to every saved Stage 1 window. This produces `y_pred_mbps`, `y_true_mbps`, elapsed time, bucket position, and metadata for every decision point across every trace.



7. Stage 2 dataset builder

The builder revisits each test at a 500 ms decision stride, rescoreing every decision window with Stage 1, computing absolute and relative error, and constructing the permanent-safe suffix oracle used for Stage 2 supervision.



8. Stage 2 Transformer

The Stage 2 model takes the full normalized trace `[100,13]` together with a prefix mask, emits a stop-versus-continue logit for each decision prefix, tunes the operating threshold on validation, and evaluates policy-level early termination behavior.

2.2 What the Original Input Looks Like

The implemented pipeline starts from exact public feature exports under `exports_exact_public/`. Each exported CSV row corresponds to one 100 ms bucket of one speed test. The reconstruction target is the final throughput that would have been obtained from the full test.

The implemented feature set has exactly **13** channels per bucket:

#	Feature name	Meaning
1	inst_throughput_mbps	Instantaneous throughput over the bucket.
2	cumavg_throughput_mbps	Cumulative average throughput up to that time.
3	pipe_full_samples	Public-data proxy for BBR pipe-full behavior.
4	mean_rtt_us	Mean RTT in microseconds over the bucket.
5	std_rtt_us	RTT standard deviation over the bucket.
6	mean_snd_cwnd	Mean sender congestion window.
7	std_snd_cwnd	Standard deviation of sender congestion window.
8	mean_bytes_in_flight	Mean bytes in flight.
9	std_bytes_in_flight	Standard deviation of bytes in flight.
10	mean_total_retrans	Mean retransmission count.
11	std_total_retrans	Standard deviation of retransmission count.
12	mean_dsack_dups	Mean duplicate selective ACK count proxy.
13	std_dsack_dups	Standard deviation of duplicate selective ACK count proxy.

Each test is represented on a fixed **100-bucket** axis, corresponding to **10 seconds** of trace duration at 100 ms resolution. So the canonical per-test dense tensor before any Stage 1 or Stage 2 reshaping is:

$$X_{\text{raw}} \in \mathbb{R}^{100 \times 13}$$

and the scalar regression target is:

$$y_{\text{true}} \in \mathbb{R}$$

2.3 Step 1: Raw Shard Construction

The script `build_exact_public_shards.py` groups CSV rows by UUID, sorts them by `bucket_100ms`, and materializes one dense test tensor per speed test. Missing buckets are first represented by a binary mask and then densified.

Shape changes in Step 1

Stage	Shape	Operation
CSV rows for one test	variable number of rows	One row per observed bucket. The test may not have all 100 buckets populated.
Sparse bucket assembly	[100, 13] + [100] mask	Observed rows are placed into their bucket index; missing buckets stay empty; <code>bucket_mask[t]=1</code> only if bucket <code>t</code> exists in the export.
Forward-fill densification	[100, 13]	Missing buckets are filled by propagating the most recent observed bucket forward; leading empty buckets copy the first observed bucket backward.
Saved raw shard batch	<code>x: [N, 100, 13]</code>	Many tests from one CSV file are stacked into a single NPZ shard.

Saved raw-shard fields

For one NPZ shard under `artifacts_exact_public/raw_shards/<split>/`, the key fields are:

- `x`: `[N,100,13] float32`
- `bucket_mask`: `[N,100] bool-like mask`
- `observed_bucket_count`: `[N]`
- `y_true_mbps`: `[N]`
- `uuid`, `date`, `speed_tier`, `test_time`

2.4 Step 2: Train-Only Normalization

The script `compute_exact_public_train_stats.py` computes train-only feature statistics across the raw train shards.

The saved train statistics in this repository were computed over:

- **800,000** train tests
- **80,000,000** train buckets

For each feature channel f , the script computes:

$$\mu_f = \frac{1}{M} \sum x_f, \quad \sigma_f = \sqrt{\max\left(\frac{1}{M} \sum x_f^2 - \mu_f^2, 0\right)}$$

with a floor so that extremely small standard deviations are clamped away from zero.

Then `materialize_normalized_exact_public_shards.py` applies:

$$X_{\text{norm}}[t, f] = \frac{X_{\text{raw}}[t, f] - \mu_f}{\sigma_f}$$

independently for every bucket t and feature f .

After this step, the shape does *not* change:

$$[100, 13] \rightarrow [100, 13]$$

Only the numerical scale changes.

2.5 Step 3: Deterministic UUID-Level Train/Val Split

The script `make_stage1_uuid_split.py` creates a deterministic UUID-level split from the normalized train split only.

The implemented split policy is:

- operate only on the original train population
- stratify by `speed_tier`

- assign 10% of UUIDs in each tier to validation
- leave `test` and `robustness` untouched

In the current repository state, that produces:

- **720,000** train UUIDs
- **80,000** val UUIDs
- exactly **144,000 train** and **16,000 val** UUIDs in each of the five speed tiers

2.6 Step 4: Stage 1 Window Materialization

Stage 1 does *not* consume the full `[100,13]` trace directly. Instead, it consumes a **2-second lookback window** ending at some bucket t .

Window construction

The script `build_stage1_regression_windows.py` uses:

- window size: 20 buckets = 2 seconds
- stride: 1 bucket = 100 ms
- output format: flattened vectors
- default decision source: observed buckets only

For an end bucket t , the Stage 1 lookback tensor is:

$$W_t \in \mathbb{R}^{20 \times 13}$$

If $t < 19$, the script pads on the left by repeating the *latest available bucket in the partial window*. That padding detail was explicitly corrected in the repository so that the pad uses the last seen bucket rather than the first bucket.

Shape changes in Stage 1 preprocessing

Representation	Shape	Meaning
Normalized full test	<code>[100,13]</code>	Entire dense test history after train-only normalization.
Stage 1 temporal window at end bucket t	<code>[20,13]</code>	Last 20 buckets ending at t , padded if needed.
Flattened Stage 1 feature vector	<code>[260]</code>	Direct concatenation of the 20×13 window.
Window shard batch	$\mathbf{x}: [\mathbf{N}, 260]$	Many decision windows stacked for XGBoost training.

The materialized Stage 1 corpus currently contains:

- **20,715,228** train windows
- **2,304,715** validation windows
- **feature dimension = 260**

2.7 Step 5: Implemented Stage 1 Regressor

Architecture

Stage 1 is not a neural network in this repository. It is an XGBoost regressor trained on flattened [260] vectors.

Stage 1, step A: input window

Input to Stage 1 is one normalized lookback window of shape [20,13]. It represents the most recent 20 buckets, or 2 seconds of history, ending at decision bucket t .



Stage 1, step B: flattening

The window is reshaped from [20,13] to [260]. This is only a deterministic reshape. No convolution, attention, or learned projection is applied before the regressor.



Stage 1, step C: XGBoost regression ensemble

The flattened vector enters a histogram-based gradient-boosted tree ensemble. The implementation uses XGBoost with `tree_method=hist` and external-memory quantile matrices for the large training run.



Stage 1, step D: scalar output

The output is a single scalar regression prediction $\hat{y}_{t,\text{final}} \in \mathbb{R}$, interpreted as the predicted full-test throughput in Mbps if the test were allowed to run to completion.

Exact Stage 1 training configuration

The Stage 1 training script `train_stage1_xgboost.py` uses the following implemented defaults:

Hyperparameter	Value	Role
<code>objective</code>	<code>reg:squarederror</code>	Scalar throughput regression
<code>tree_method</code>	<code>hist</code>	Histogram tree builder
<code>learning_rate</code>	0.03	Shrinkage per boosting round
<code>max_depth</code>	7	Tree depth
<code>min_child_weight</code>	10.0	Child split regularization
<code>subsample</code>	0.8	Row sampling
<code>colsample_bytree</code>	0.8	Feature sampling
<code>reg_lambda</code>	1.0	L2 regularization
<code>max_bin</code>	256	Histogram bin count
<code>nthread</code>	8 in final run	CPU parallelism
<code>num_boost_round</code>	1500	Maximum rounds
<code>early_stopping_rounds</code>	50	Validation-based stopping rule
<code>eval metrics</code>	<code>rmse, mae</code>	Validation tracking

The completed full run used `nthread = 8`, ran all 1500 rounds, and ended with `best_iteration = 1499`, meaning early stopping did not trigger.

Stage 1 scoring output

The script `score_stage1_xgboost.py` applies the trained regressor to every Stage 1 window shard and writes:

- `y_pred_mbps`
- `y_true_mbps`
- `uuid, speed_tier, test_time`
- `end_bucket, elapsed_ms`
- `observed_buckets_seen, last_observed_bucket`

These scored outputs are both a direct regression benchmark and the basis for Stage 2 reasoning.

2.8 Step 6: Implemented Stage 2 Dataset Builder

Stage 2 is where the pipeline transitions from “predict the final speed from a short window” to “decide when it is safe to stop the test early”.

The script `build_stage2_transformer_dataset.py` does *not* reuse the old Stage 1 window shards directly. Instead, it starts again from the normalized full-history traces.

Conceptual role of the Stage 2 dataset builder

For each full test $X \in \mathbb{R}^{100 \times 13}$:

1. choose a set of **decision buckets** at 500 ms stride
2. for each decision bucket, build the corresponding Stage 1 lookback window
3. run the trained Stage 1 regressor on that window
4. compare the Stage 1 prediction to the test’s final throughput
5. derive an *instantaneous safe* indicator
6. derive a stricter *permanent-safe suffix oracle*
7. write one Stage 2 training example per test, with the full history and all decision-table metadata

Decision timeline

The implemented defaults are:

- Stage 1 lookback window size: 20 buckets
- decision stride: 5 buckets = 500 ms

- minimum decision bucket: 4
- maximum sequence length: 100 buckets

So the valid decision positions are:

$$t \in \{4, 9, 14, 19, \dots, 99\}$$

Therefore the maximum number of decisions per test is:

$$\text{max_decisions_per_test} = 20$$

Shape changes inside the Stage 2 dataset builder

Stage 2 dataset, step A: full normalized trace

The builder starts from one normalized full-history trace `x_full: [100, 13]`. This keeps the complete 10-second trace rather than only a short lookback window.



Stage 2 dataset, step B: decision schedule

The builder defines up to 20 decision slots at a 500 ms stride. Valid slots are tracked by `decision_valid_mask: [20]`, because shorter tests may expose fewer usable decisions.



Stage 2 dataset, step C: Stage 1 rescoring at each decision

For every valid decision position d , the builder extracts the corresponding Stage 1 lookback window, reshapes it from `[20, 13]` to `[260]`, and runs the trained XGBoost regressor to get a final-throughput prediction for that decision point.



Stage 2 dataset, step D: decision table

The builder assembles one per-decision table containing `y_pred_mbps[20]`, `abs_error[20]`, `relative_error[20]`, elapsed time, observed buckets seen, and the `instantaneous_safe[20]` flag derived from the epsilon rule.



Stage 2 dataset, step E: permanent-safe suffix oracle

The builder finds the earliest decision index d^* such that every later decision is also safe. This converts local safety into a monotonic policy target, so the model learns the earliest point after which the trace remains safe forever.



Stage 2 dataset, step F: saved supervised sample

The final saved sample keeps the full trace `[100, 13]` and the full decision table together. The label side includes `stop_label`, `continue_label`, and `is_oracle_stop_window`, which are all derived from the permanent-safe suffix oracle.

Exact oracle logic

For each decision d , the builder computes:

- $\hat{y}_d$: Stage 1 prediction at decision d
- y_{true} : final throughput of the full test
- $|\hat{y}_d - y_{\text{true}}|$: absolute error
- $\frac{|\hat{y}_d - y_{\text{true}}|}{|y_{\text{true}}|}$: relative error

Then it defines:

$$\text{instantaneous_safe}[d] = 1 \quad \text{iff} \quad \text{relative_error}[d] \leq \epsilon$$

for the relative-error setting.

The permanent-safe suffix oracle is stricter:

$$d^* = \min \{d : \text{instantaneous_safe}[k] = 1 \quad \forall k \geq d\}$$

Then:

$$\text{stop_label}[d] = \begin{cases} 1 & d \geq d^* \\ 0 & d < d^* \end{cases}$$

This is important because Stage 2 is not trained merely to identify a safe point. It is trained to identify the *earliest point after which the rest of the trace remains safe*.

Saved Stage 2 tensor contract

An actual Stage 2 train shard in this repository has the following implemented shapes:

Field	Shape	Role
<code>x_full</code>	[20000, 100, 13]	Full normalized traces
<code>bucket_mask</code>	[20000, 100]	Observed-bucket mask
<code>decision_valid_mask</code>	[20000, 20]	Which decision slots are real
<code>decision_end_bucket</code>	[20000, 20]	Bucket index of each decision slot
<code>decision_elapsed_ms</code>	[20000, 20]	Elapsed time per decision
<code>decision_observed_buckets_seen</code>	[20000, 20]	Number of observed buckets so far
<code>y_pred_mbps</code>	[20000, 20]	Stage 1 prediction at each decision
<code>abs_error_mbps</code>	[20000, 20]	Absolute error at each decision
<code>relative_error</code>	[20000, 20]	Relative error at each decision
<code>instantaneous_safe_window</code>	[20000, 20]	Safety flag at each decision
<code>stop_label</code>	[20000, 20]	Monotonic suffix label for Stage 2 training
<code>continue_label</code>	[20000, 20]	Complement label
<code>is_oracle_stop_window</code>	[20000, 20]	Single entry point marker d^*
<code>oracle_stop_found</code>	[20000]	Whether any permanent-safe suffix exists
<code>y_true_mbps</code>	[20000]	Final throughput target for the test

2.9 Step 7: Implemented Stage 2 Transformer

What the model consumes during training

Although each NPZ row contains the full test and the full decision table, the training loop converts *each valid decision* into one supervised decision example.

For one decision example, the training loop constructs:

- `x_full`: [100, 13]
- `history_lengths`: scalar $h = t + 1$, where t is the chosen decision end bucket
- `attention_mask`: [100] with ones up to the prefix length h
- `label`: `stop_label[t]` or `is_oracle_stop_window[t]`

At batch size B , the forward pass sees:

$$\begin{aligned}
 \mathbf{x_full} &\in \mathbb{R}^{B \times 100 \times 13} \\
 \mathbf{attention_mask} &\in \{0, 1\}^{B \times 100} \\
 \mathbf{history_lengths} &\in \mathbb{N}^B
 \end{aligned}$$

Exact implemented Transformer architecture

The implemented Stage 2 model class is `Stage2Transformer` in `train_stage2_transformer.py`.

Stage 2 Transformer, layer 1: masked full-prefix input

Each training example provides a full trace tensor $[B, 100, 13]$. An attention mask marks which buckets are visible for the current decision prefix, so the model only reasons over the history that would really have been available at that stopping time.

**Stage 2 Transformer, layer 2: input projection**

Every 13-dimensional bucket vector is projected independently with a linear map from 13 channels to 128 channels. The sequence shape changes from $[B, 100, 13]$ to $[B, 100, 128]$.

**Stage 2 Transformer, layer 3: learned positional encoding**

A learned position tensor of shape $[1, 100, 128]$ is added elementwise to the projected sequence so the model can distinguish early, middle, and late buckets.

**Stage 2 Transformer, layer 4: Transformer encoder stack**

The sequence enters an 8-layer Transformer encoder with 8 attention heads per layer, feed-forward width 512, dropout 0.1, and pre-normalization enabled through the PyTorch encoder layer configuration used in the repository.

**Stage 2 Transformer, layer 5: prefix pooling**

After encoding, the model selects the hidden state at the last valid prefix position, namely `hidden[batch, history_length - 1]`. This produces one pooled vector $[B, 128]$ for the current stopping decision.

**Stage 2 Transformer, layer 6: normalization and classification head**

The pooled vector passes through LayerNorm, then through a small multilayer perceptron: linear 128 to 128, GELU, dropout, and linear 128 to 1. The result is one scalar stop logit for the current decision prefix.

**Stage 2 Transformer, final output**

Applying a sigmoid converts the scalar logit into a stop probability. Thresholding that probability yields the final stop-versus-continue decision used in policy evaluation.

Per-layer tensor shapes

Layer	Output shape	Notes
Input full trace	[B,100,13]	Full test trace always passed in; masking controls visible prefix.
Input projection	[B,100,128]	Linear map from 13 features to model width 128.
Add learned position embeddings	[B,100,128]	Same shape; adds position signal for each bucket.
Transformer encoder layer 1	[B,100,128]	Multi-head self-attention + feed-forward block.
Transformer encoder layer 2	[B,100,128]	Same width, deeper temporal mixing.
Transformer encoder layer 3	[B,100,128]	Same width.
Transformer encoder layer 4	[B,100,128]	Same width.
Transformer encoder layer 5	[B,100,128]	Same width.
Transformer encoder layer 6	[B,100,128]	Same width.
Transformer encoder layer 7	[B,100,128]	Same width.
Transformer encoder layer 8	[B,100,128]	Final contextualized sequence.
Last-valid-token gather	[B,128]	Uses <code>history_lengths</code> to pick the prefix endpoint.
LayerNorm	[B,128]	Stabilizes pooled decision embedding.
Head hidden layer	[B,128]	Linear + GELU + dropout.
Final output layer	[B,1] then squeeze	One scalar stop logit per decision example.

Exact implemented training setup

The implemented defaults are:

- epochs: 5
- loss: `BCEWithLogitsLoss`
- positive class weight: inferred from dataset summary unless set manually
- optimizer: `Adam`
- learning rate: 10^{-3}
- weight decay: 0
- gradient clipping norm: 1.0

The final valid local GPU run for `epsilon = 10` used:

- physical batch size: 1024
- gradient accumulation steps: 4
- effective batch size: 4096
- best epoch: 3
- best threshold: 0.25

Thresholding and policy evaluation

After the model outputs a stop probability for every decision row, evaluation groups rows back by `(uuid,test_time)`. Then:

1. scan decisions in temporal order
2. take the first decision with probability $\geq$ threshold
3. if no decision crosses the threshold, continue to full test completion

This produces policy-level metrics such as:

- emitted stop rate
- within-epsilon rate
- mean and median stop elapsed time
- savings versus the full test
- excess delay versus the oracle stopping point

2.10 Implemented Baseline Summary in One Page

Original input	Per-test trace exported as bucketed 100 ms measurements with 13 features per bucket, densified to [100,13].
Preprocessing	Forward-fill densification, train-only normalization, UUID-level train/val split.
Stage 1 input	Last 20 buckets of the normalized trace, shape [20,13], flattened to [260].
Stage 1 model	XGBoost regressor, <code>max_depth=7</code> , <code>learning_rate=0.03</code> , <code>tree_method=hist</code> , 1500 rounds.
Stage 1 output	Scalar final-throughput prediction for a chosen end bucket.
Stage 2 input data	Full normalized trace [100,13] plus a per-test decision table with up to 20 decision points.
Stage 2 labels	Permanent-safe suffix oracle, stored as monotonic <code>stop_label</code> .
Stage 2 model	Transformer encoder with <code>d_model=128</code> , <code>num_heads=8</code> , <code>num_layers=8</code> , <code>ff_dim=512</code> .
Stage 2 output	Stop / continue logit for one prefix decision.

3 Part II: Proposed Foundation Model Architecture

3.1 Why We Move Beyond the Baseline

The implemented baseline reproduces the two-stage paper structure:

- Stage 1: short-window regressor
- Stage 2: stop-policy classifier on full-history prefixes

That is a strong engineering baseline, but it is not yet the actual project contribution promised in the proposal. The proposal’s next step is a **pretrained foundation model for multivariate network traces** that can support multiple downstream tasks, not only early stopping.

So from this point onward, the baseline becomes the comparison point, while the **new model** becomes the main research architecture.

3.2 Design Goal for the New Model

The new model should satisfy four constraints:

1. it must consume the same underlying trace family we already prepared, so it can reuse current artifacts
2. it must learn reusable sequence representations, not task-specific heuristics
3. it must support at least three downstream tasks:
 - early stopping
 - final throughput prediction
 - speed-grade classification
4. it must preserve comparability with the baseline and the original paper

To meet those constraints, the proposed next model is a **patch-based pretrained Transformer encoder** over the normalized [100,13] speed-test trace.

3.3 Fixed Implementation Scope for the Foundation Model

The baseline freeze does not block the foundation-model work. Pretraining, throughput regression, and speed-grade classification use normalized shards directly. Early-stopping fine-tuning uses only the valid $\epsilon = 10$ Stage 2 dataset and is compared against the frozen $\epsilon = 10$ Transformer baseline. Early-stopping experiments for other epsilon values are out of scope unless those Stage 2 datasets are rebuilt under a matching XGBoost runtime in a future extension.

3.4 Implementation Decisions Before Coding

The following decisions are fixed before implementation so the architecture is unambiguous:

Decision	Specification
Bucket-mask handling	Patchification carries the original <code>bucket_mask</code> . A patch is considered valid if at least one of its five buckets was observed before forward-fill. Invalid patches are passed to the Transformer as masked positions through <code>src_key_padding_mask</code> ; the <code>[CLS]</code> token is always valid.
Pretraining split	Self-supervised pretraining uses only the Stage 1 train UUID subset, i.e. the 720,000 train UUIDs. Validation, test, and robustness UUIDs are not used for pretraining, preventing downstream evaluation leakage.
Masked-patch ratio	Objective A uses a 50 percent patch-mask ratio over valid patches. With only 20 patch tokens per test, this produces a meaningful reconstruction task while still leaving observed context. At least one valid patch remains visible.
Next-patch attention pattern	Objective B feeds only the first p visible patches and predicts patch $p + 1$. Future patches are not present in the input sequence, so no future information can leak through attention.
Pooling choice	The implemented Stage 2 baseline pools the last valid bucket token. The foundation model deliberately uses a <code>[CLS]</code> token so the same sequence-level embedding can support pretraining, throughput regression, early stopping, and speed-grade classification.

3.5 Proposed Input Representation

The starting point remains the normalized dense trace:

$$X \in \mathbb{R}^{100 \times 13}$$

Instead of feeding individual buckets directly as 100 separate tokens, the proposed foundation model groups buckets into **non-overlapping 500 ms patches**. This choice is deliberate:

- it reduces token count
- it aligns the representation with the baseline Stage 2 decision stride
- it makes pretraining and downstream prefix reasoning cheaper

Patchification

Patch size:

$$P = 5 \text{ buckets} = 500 \text{ ms}$$

Number of patches per test:

$$100/5 = 20$$

So the full trace becomes:

$$X_{\text{patch}} \in \mathbb{R}^{20 \times 5 \times 13}$$

Flatten each patch:

$$X_{\text{flat-patch}} \in \mathbb{R}^{20 \times 65}$$

because $5 \times 13 = 65$.

3.6 Exact Proposed Foundation Encoder

The next model architecture proposed here is:

Component	Output shape	Purpose
Patchify [100, 13] into 20 patches of 5 buckets	[20, 5, 13]	Convert per-bucket trace to coarser temporal tokens.
Flatten each patch	[20, 65]	Turn one patch into one feature vector.
Patch embedding MLP: Linear 65 $\rightarrow$ 128, GELU, Linear 128 $\rightarrow$ 256	[20, 256]	Learn a richer token representation than raw flattening.
Prepend learned [CLS] token	[21, 256]	Global summary token for sequence-level downstream tasks.
Add learned positional embedding	[21, 256]	Preserve patch order.
Transformer encoder, 8 layers, 8 heads, FF = 1024, dropout = 0.1	[21, 256]	Shared foundation encoder.
CLS projection head(s)	task-dependent	Produce outputs for downstream tasks.

Per-layer shape walkthrough

Layer	Shape	Explanation
Input normalized trace	[B, 100, 13]	Same dense trace already used in the baseline.
Patchify into 20 windows	[B, 20, 5, 13]	Each token spans 500 ms.
Flatten each patch	[B, 20, 65]	No learning yet; pure reshape.
Linear 65 $\rightarrow$ 128	[B, 20, 128]	First embedding projection.
GELU	[B, 20, 128]	Nonlinearity.
Linear 128 $\rightarrow$ 256	[B, 20, 256]	Final patch token width.
Prepend [CLS] token	[B, 21, 256]	One learnable sequence-summary token.
Add positional embedding	[B, 21, 256]	Learned sequence position encoding.
Transformer encoder layer 1	[B, 21, 256]	Self-attention over all patches and CLS.
Transformer encoder layer 2	[B, 21, 256]	Same width, deeper context.
Transformer encoder layer 3	[B, 21, 256]	Same width.
Transformer encoder layer 4	[B, 21, 256]	Same width.
Transformer encoder layer 5	[B, 21, 256]	Same width.
Transformer encoder layer 6	[B, 21, 256]	Same width.
Transformer encoder layer 7	[B, 21, 256]	Same width.
Transformer encoder layer 8	[B, 21, 256]	Final reusable representation.
CLS extraction	[B, 256]	Sequence-level embedding for downstream heads.

3.7 Proposed Pretraining Objectives

The proposal explicitly calls for self-supervised pretraining. The recommended first version should use objectives that are easy to implement from the current dataset and naturally aligned with network traces. All pretraining is restricted to the 720,000 train UUIDs, not validation, test, or robustness UUIDs.

Objective A: Masked patch reconstruction

Input:

$$[B, 20, 65] \rightarrow [B, 21, 256] \rightarrow \text{encoder output}$$

Randomly mask 50 percent of valid patch tokens. A patch is valid when at least one of its five original buckets was observed before forward-fill. Invalid patches are excluded from the reconstruction loss and masked in attention. For each masked valid patch token, attach a reconstruction head:

$$256 \rightarrow 128 \rightarrow 65$$

Then reshape:

$$65 \rightarrow [5, 13]$$

This teaches the encoder to infer missing future or missing local context from the rest of the trace. Because the trace has only 20 patch tokens, the 50 percent mask ratio is intentionally stronger than a BERT-style 15 percent token mask.

Objective B: Next-patch forecasting

Given the first p visible patches, use the [CLS] output to predict patch $p + 1$:

$$\text{CLS}[256] \rightarrow 128 \rightarrow 65$$

The input sequence for this objective contains only the first p patches plus [CLS]; patches after p are not included in the encoder input. This directly encourages prefix-to-future reasoning without letting attention see future tokens.

Why these two objectives first

They fit the current data without new labeling, and they prepare the encoder for:

- reconstructing missing network dynamics
- predicting what happens next from an incomplete prefix
- transferring the representation to stopping, regression, and classification

3.8 Downstream Heads for the Foundation Model

The pretrained encoder is shared. Only the task head changes.

3.8.1 Head 1: Early stopping

For early stopping, we create prefixes of length $p \in \{1, \dots, 20\}$ patches. That means the model sees only the first p patch tokens plus the [CLS] token. Early-stopping fine-tuning and evaluation are restricted to $\epsilon = 10$, matching the frozen baseline.

Foundation model early-stop head, step A: patch-prefix input

For early stopping, the model receives only the first p patch tokens, where $1 \leq p \leq 20$. The patch-prefix tensor has shape $[B, p, 65]$ before embedding.

**Foundation model early-stop head, step B: shared encoder**

The prefix passes through the shared patch-embedding stack, receives a prepended [CLS] token, and then goes through the shared 8-layer Transformer encoder. The encoded output has shape $[B, p+1, 256]$.

**Foundation model early-stop head, step C: CLS pooling**

The [CLS] embedding is extracted as `encoder[:, 0, :]`. This yields one sequence-level representation of shape $[B, 256]$ for the prefix. This is a deliberate difference from the implemented Stage 2 baseline, which pools the last valid bucket token.

**Foundation model early-stop head, step D: binary stop head**

The [CLS] vector is passed through a small head with linear 256 to 128, GELU, and linear 128 to 1. After sigmoid, the output becomes $\hat{p}_{\text{stop}} \in [0, 1]$.

Output:

$$\hat{p}_{\text{stop}} \in [0, 1]$$

Prefix-to-baseline alignment

Foundation prefix length $p \in \{1, \dots, 20\}$ corresponds exactly to implemented Stage 2 decision index $d = p - 1$. Patch $p - 1$ covers buckets $5(p - 1)$ through $5(p - 1) + 4$, so it ends at bucket $5p - 1$. The implemented Stage 2 decision with index d also ends at bucket $5d + 4 = 5p - 1$. Therefore, during $\epsilon = 10$ early-stop fine-tuning, the supervised target for foundation-model prefix p is `stop_label[p-1]` from the frozen $\epsilon = 10$ Stage 2 dataset.

3.8.2 Head 2: Final throughput regression

Use the same [CLS] token:

$$256 \rightarrow 128 \rightarrow 1$$

Output:

$$\hat{g}_{\text{final}} \in \mathbb{R}$$

This becomes the direct learned alternative to Stage 1 XGBoost.

This task uses normalized shards plus `y_true_mbps`; it does not depend on rebuilding any stale Stage 2 epsilon datasets.

3.8.3 Head 3: Speed-grade classification

Use the same [CLS] token:

$$256 \rightarrow 128 \rightarrow 5$$

with softmax over the five currently used speed tiers:

$$\{0-25, 25-100, 100-200, 200-400, 400+\}$$

This task uses normalized shards plus `speed_tier`; it does not depend on the baseline Stage 2 epsilon sweep.

3.9 Full Proposed Foundation-Model Flow

Foundation model, step 1: normalized base trace

The foundation model starts from the same normalized trace representation already used by the baseline, namely one tensor of shape `[100,13]` per test.



Foundation model, step 2: patch tokenization

The 100-bucket sequence is split into 20 non-overlapping patches of 5 buckets each. The shape changes as `[100,13] -> [20,5,13] -> [20,65]`. Each patch token therefore summarizes 500 ms of trace history.



Foundation model, step 3: patch embedding MLP

Each 65-dimensional patch token is mapped through a learned embedding stack: linear 65 to 128, GELU, and linear 128 to 256. The patch sequence therefore becomes `[20,256]` before sequence modeling.



Foundation model, step 4: CLS token and position encoding

A learned [CLS] token of shape `[1,256]` is prepended, producing `[21,256]`. A learned positional embedding of the same sequence length is then added so the encoder knows the order of patches.



Foundation model, step 5: shared Transformer encoder

The token sequence passes through an 8-layer Transformer encoder with model width 256, 8 attention heads, feed-forward width 1024, and dropout 0.1. The encoder output remains `[21,256]`.



Foundation model, step 6: reusable shared representation

The encoded [CLS] token is treated as the sequence-level trace embedding. This gives one reusable representation `[256]` that can be consumed by multiple heads instead of training separate models per task.

**Foundation model, step 7: pretraining and downstream heads**

During self-supervised pretraining, the encoder feeds masked-patch reconstruction and next-patch forecasting heads. During downstream fine-tuning, the same encoder feeds an early-stop binary head, a final-throughput regression head, and a 5-way speed-grade classification head.

3.10 Exact Input and Output Contracts for the Proposed Model

Pretraining mode

Item	Shape	Meaning
Normalized trace batch	[B, 100, 13]	Raw reusable trace input.
Bucket mask batch	[B, 100]	Observed-bucket mask used to derive patch validity.
Patchified batch	[B, 20, 65]	20 patch tokens per test.
Patch validity mask	[B, 20]	Patch is valid if at least one source bucket was observed.
Encoder output	[B, 21, 256]	20 patch tokens + CLS token.
Masked reconstruction output	[B, 20, 65]	Reconstruct masked patch vectors.
Next-patch output	[B, 65] [B, 20, 65]	or Predict the next patch from visible prefix.

Fine-tuning mode

Task	Output shape	Target
Early stopping	[B, 1]	Safe-to-stop vs continue at a given prefix.
Final throughput regression	[B, 1]	Final throughput in Mbps.
Speed-grade classification	[B, 5]	Tier classification logits.

4 Implemented Baseline vs Proposed Foundation Model

Dimension	Implemented baseline	Proposed foundation model
Base input	Dense test trace [100,13]	Same dense trace [100,13]
Main tokenization	Stage 1: [20,13] flattened to [260]; Stage 2: per-bucket sequence [100,13]	Patch tokens: 20 patches of size 5 x 13, flattened to [20,65]
Core predictor	XGBoost regressor + Transformer stop classifier	One pretrained shared Transformer encoder
Representation reuse	Minimal; Stage 1 and Stage 2 are separate	Explicitly shared across tasks
Primary output	Stage 1 throughput, Stage 2 stop probability	Shared trace embedding + task-specific outputs
Pretraining	None	Self-supervised masked reconstruction and next-patch prediction
Downstream tasks	Throughput prediction and stopping	Throughput prediction, stopping at $\epsilon = 10$, speed-grade classification, possible uncertainty extensions
Evaluation scope	Frozen $\epsilon = 10$ Stage 2 line for internal early-stopping baseline	Early stopping compared at $\epsilon = 10$; throughput regression and speed-grade classification use their own downstream metrics

5 What Is Already Implemented vs What Comes Next

Already implemented in repo	Exact public-data preprocessing, train-only normalization, UUID split, Stage 1 window materialization, Stage 1 XGBoost training and scoring, Stage 2 dataset materialization, Stage 2 Transformer training and offline threshold sweep.
Valid frozen baseline right now	Stage 1 full pipeline and the <code>epsilon = 10</code> Stage 2 line.
Descoped baseline work	The multi-epsilon baseline sweep is descoped; foundation-model evaluation is anchored at $\epsilon = 10$ for early stopping.
Not yet implemented	The pretrained foundation model described in Part II.
Immediate next engineering work	Build the patchifier, patch embedding MLP, and 8-layer encoder skeleton; run masked-patch pretraining on train UUIDs; then fine-tune throughput regression, $\epsilon = 10$ early stopping, and speed-grade classification heads.

6 Final Summary

The implemented baseline reproduction in this repository has a very clear data flow:

- CSV bucket rows
- raw dense traces
- normalized traces

- Stage 1 windows
- Stage 1 XGBoost predictions
- Stage 2 decision tables
- Stage 2 Transformer policy decisions

At each stage, the representation changes in a specific and recoverable way:

- raw per-bucket rows become dense $[100, 13]$ traces
- dense traces become Stage 1 windows $[20, 13]$ and then flat vectors $[260]$
- Stage 1 scalar predictions become per-decision error tables $[20]$
- full traces plus decision tables become the supervision source for the Stage 2 Transformer

The next model should not replace this understanding. It should build on it. That is why the proposed foundation model keeps the same normalized trace as input, but converts it into reusable patch tokens and a shared Transformer representation that can support multiple downstream tasks from one pretrained encoder.